PPL Assignment 2 - Question 2.d: Class to Proc Transformation

Syntactic Transformation

May 20, 2026

Question 1.1

No, there is no program in L1 that cannot be transformed to an equivalent program in L11. In other words, **every program in L1 can be transformed into an equivalent program in L11**.

Explanation:
The L1 language does not contain user-defined functions (`lambda`), local variables, or side-effects other than the `define` operation itself. In L1, the `define` structure simply binds a name to a primitive value or the result of an expression. Since there are no side-effects or state changes during runtime, we can take any program that uses `define` and apply **substitution**: wherever the defined variable name appears, we replace it directly with the evaluated expression or value it points to.

Question 1.2

No, every L2 program can be transformed into an equivalent L21 program.

Unlike L1, L2 includes lambda expressions, which allow variable bindings through function abstraction and application. For simple non-recursive bindings, any use of `define` can be directly replaced by wrapping the body in a lambda expression and applying it immediately. For example:
`(define x 5) (+ x 3)` can be transformed into `((lambda (x) (+ x 3)) 5)`.

The main challenge in L2 is **recursive procedures**, where a function needs to reference its own name. However, even without `define`, recursion can be successfully achieved in L21 by using the **Y-combinator** pattern (or the self-passing function technique), where the anonymous lambda expression passes itself as an additional argument to achieve self-reference.

Question 1.3

No, every program in L2 can be transformed into an equivalent program in L22.

Explanation:
The syntactic restrictions of L22 reflect the foundational rules of pure **Lambda Calculus**, which is Turing-Complete. Any L2 program can be systematically rewritten to satisfy L22 requirements using **Currying**: Functions with multiple arguments are transformed into a nested chain of single-parameter functions.
Example: `(lambda (x y) (* x y))` becomes `(lambda (x) (lambda (y) (* x y)))`.

Question 1.4

Yes, there is a program in L2 that cannot be transformed to an equivalent program in L23.

Contradictory Example:

Listing 1: Example of program not transformable to L23

```
(define apply-conditionally
  (lambda (cond? f g x)
    (if cond? (f x) (g x))))
```

Explanation:
In L2, functions are first-class citizens and can be passed as arguments. The crucial point is that the exact function passed to `f` or `g` might only be determined dynamically at runtime based on other conditions

or user inputs. Since L23 strictly forbids passing functions as arguments, and we cannot predict this dynamic runtime behavior in advance, we cannot statically "hardcode" or inline the specific function calls ahead of time.

Furthermore, we cannot simulate this dynamic behavior using workarounds. In L2, functions act as **closures** that capture variables from their lexical environment. Because L2 lacks built-in complex data structures (like arrays or lists) to save this captured state, it relies entirely on higher-order functions to represent and store dynamic data. Without the ability to pass functions, L23 has no way to pack, store, or evaluate these closures at runtime, making the transformation impossible without losing the program's core logic.

Question 1.5

Special forms are required because standard primitive operators rigidly evaluate **all** of their arguments before the operator itself is applied (applicative-order evaluation). We cannot define control-flow structures or environment bindings as primitive operators because they require custom evaluation rules—they need the power to control *if*, *when*, or *how* their arguments are evaluated.

Example:
Consider the `if` expression: `(if (= x 0) 0 (/ 5 x))`.
If `if` were defined simply as a primitive operator, the language evaluator would evaluate all of its arguments first before applying the `if` logic. If `x` happens to be `0`, the program would attempt to evaluate the third argument `(/ 5 0)` and immediately crash with a **Division by Zero** error, defeating the entire purpose of the condition.

Question 1.6

No, a function body with multiple expressions is not required in pure functional programming.

Explanation:

- **Pure Functional Programming:** Pure functions have no side-effects (like altering memory or printing). If a function body has multiple expressions, the evaluator calculates the earlier expressions and immediately discards their results, returning only the final expression. Since earlier expressions don't change any state, evaluating them is useless; a single expression is entirely sufficient.
- **Where is it useful?** Multi-expression bodies are essential in **impure or imperative languages** that support side-effects. In such languages, early expressions perform sequential, meaningful actions (like updating a variable or printing data) before the final value is returned.
- **What about L3?** L3 introduces local bindings (`let`) and data structures, but it remains a pure functional language with no side-effects. Instead of writing a sequence of separate expressions to calculate intermediate steps, L3 handles this purely by binding intermediate results to local variables using a single `let` expression. Therefore, a multi-expression body is still not required.

Question 1.7

A **Lexical Address** is a static coordinate system used to replace variable names in an Abstract Syntax Tree (AST) with exact pointers to where they are defined. This optimizes the interpreter by removing the need for dynamic string-matching environments during runtime.

A bound variable's lexical address consists of two coordinates:

1. **Depth:** The number of nested scopes (e.g., `lambda` contours) the interpreter must traverse outward to reach the variable's declaration. `0` indicates the immediate current scope.
2. **Position:** The zero-based index of the variable within that specific scope's parameter list.

Example:
Consider the following nested expression:

Listing 2: Nested Lambda Expression

```
(lambda (y x) ((lambda (x) (+ x y)) (+ x z)))
```

When transformed to use lexical addresses in the format `[variable : depth position]`, it becomes:

Listing 3: Expression with Lexical Addresses

```
(lambda (y x) ((lambda (x) (+ [x : 0 0] [y : 1 0])) (+ [x : 0 1] [z : free])))
```

Question 1.8

Advantage of the PrimOp representation:
Execution efficiency and direct implementation. When primitive operators are parsed as dedicated syntactic nodes (e.g., `PrimOp("+")`), the interpreter recognizes them immediately and triggers the underlying native code. This approach completely bypasses the runtime overhead associated with searching through dynamic environments, creating closures, or handling substitution processes.

Advantage of the Closure representation:
Architectural uniformity and first-class flexibility. By predefining primitive operators as standard closures in the initial global environment (parsed simply as variable references), the interpreter treats built-in and user-defined functions exactly the same. This simplifies the core evaluation logic by eliminating special-case handling for applications, while allowing primitives to act as first-class citizens that can be seamlessly passed as arguments or bound to new variables.

Question 1.9

a. Switching from Applicative Order to Normal Order

Reasons for switching:

1. **Ensuring Termination (Halting):** In applicative order, all arguments are evaluated before the function is applied. If an unused argument contains an infinite loop or an error, the program will crash or hang. Normal order delays evaluation until the argument is actually needed, allowing the program to terminate successfully if that argument is discarded.

2. **Avoiding Unnecessary Computations:** If an argument passed to a function is never used within its body, applicative order wastes processing time evaluating it. Normal order completely skips its evaluation.

Example:

Listing 4: Applicative vs Normal Order Example

```
(define loop (lambda () (loop)))
(define f (lambda (func) 5))

(f (loop))
```

b. Switching from Normal Order to Applicative Order Evaluation

Primary Reason:
Efficiency and Avoiding Redundant Computations. In pure normal order (call-by-name), if an argument is used multiple times inside the function body, the un-evaluated expression is duplicated. Consequently, the interpreter is forced to evaluate the exact same expression multiple times during execution. Applicative order resolves this issue completely by evaluating the argument **exactly once** before entering the function body and passing a pre-calculated final value, thereby optimizing runtime and saving system resources.

Example in L3:

Listing 5: Normal vs Applicative Order Example

```
(define square (lambda (x) (* x x)))
(define expensive-calc (lambda () (+ 5 5)))

(square (expensive-calc))
```

Question 1.10

a. The Role of `valueToLitExp` in L3

Role and Purpose:
The function `valueToLitExp` is a helper function used in the L3 interpreter during the evaluation of expressions under the **Substitution Model**. Its primary role is to convert a computed runtime `Value` (such as `NumVal` or `BoolVal`) back into a literal expression (`LitExp`), which is a node type in the Abstract Syntax Tree (AST).

Why it is necessary:

- **Type Mismatch in the AST:** In the substitution model, when a function application (`AppExp`) is evaluated, the arguments are first reduced to runtime `Value` objects. The interpreter must then substitute these evaluated results into the function's body.
- **AST Expects Expressions (`Exp`):** The function body is represented as an AST composed entirely of expressions (`Exp`). You cannot directly inject a raw runtime `Value` (like `NumVal(5)`) into an AST structure that strictly expects an `Exp` subtype.
- **The Solution:** `valueToLitExp` bridges this operational gap. It takes the runtime `Value` and maps it to its corresponding AST literal expression representation (e.g., mapping `NumVal(5)` to `LitExp(5)`). Since `LitExp` is a valid `Exp`, the substitution algorithm can safely replace variable references (`VarRef`) with this literal expression node throughout the function body's AST.

b. Why `valueToLitExp` is not needed in the Normal Evaluation Strategy (`L3-normal.ts`)

In the normal order evaluation strategy (call-by-name), arguments are passed into a function **without being evaluated beforehand**.

- Instead of reducing an argument to a runtime `Value` object (like `NumVal`), the interpreter takes the raw, un-evaluated argument expression directly from the AST (which is already of type `Exp`).
- The substitution mechanism then replaces the parameter variables within the function body's AST directly with these `Exp` nodes.
- Since the process involves substituting an `Exp` with another `Exp`, the interpreter never manipulates runtime `Value` objects during the substitution phase. Therefore, converting a `Value` to a `LitExp` is completely obsolete.

c. Why `valueToLitExp` is not needed in the Environment-Model Interpreter

The environment-model interpreter **completely eliminates the substitution mechanism** from the evaluation process, keeping the AST structurally immutable during runtime.

- In the substitution model, we are forced to inject evaluated results back into the function body's AST, causing a type mismatch that requires `valueToLitExp`.
- In the environment model, when a function application is evaluated, the interpreter creates a new environment frame (a binding map). This frame maps the parameter names (strings) directly to their computed runtime `Value` objects.
- When evaluating the function body, whenever the interpreter encounters a variable reference (`VarRef`), it dynamically looks up the variable's name in the current environment and retrieves its `Value`.

- Because runtime values are stored externally in environments rather than being spliced back into the AST node tree, there is no need to transform a `Value` into a `LitExp`.

Question 1.11

a. Why Renaming is Not Required in the Environment Model

Renaming is completely unnecessary in the environment model because it **eliminates the substitution mechanism**, meaning the AST remains structurally immutable during runtime, preventing any possibility of variable capture.

- In the substitution model, evaluating code involves splicing expressions into function bodies, which risks "capturing" free variables if a local parameter happens to share the same name. To prevent this, strict renaming rules are required.
- In the environment model, when a function (closure) is defined, it captures a reference to its definition-time environment. When that closure is applied, a new environment frame is created, which links back to the closure's captured environment.
- Variable resolution is performed dynamically at runtime by traversing this static chain of environment frames. Since variables are resolved by looking up names within distinct, isolated memory scopes rather than modifying the text of the program, there is no physical overlap or textual merging of expressions. Consequently, free variables are always safely resolved in their proper lexical scope, entirely avoiding variable capture and making renaming obsolete.

b. Is Renaming Required in the Substitution Model When the Substituted Term is Closed?

No, renaming is not required in the substitution model if the term being substituted is a closed term.

- **What is a closed term?** By definition, a closed term is an expression where all variables are locally bound, meaning it contains **no free variables**.
- **When is renaming needed?** In the substitution model, renaming is only triggered to prevent "variable capture." This capture occurs exclusively when a *free* variable inside the injected expression happens to share the same name as a local parameter in the target function, causing it to be accidentally bound to the wrong scope.
- **Why is it safe here?** Since the expression we are substituting is completely closed, it has absolutely no free variables that could clash with or be captured by other parameters. Without any free variables, the risk of capture is non-existent. Therefore, the substitution is perfectly safe and precise without ever needing to activate the renaming mechanism.

Question 2.d: Transformed Program

Below is the equivalent program where the `ClassExp` representing the `circle` has been syntactically transformed into nested `ProcExp`s (lambdas) with a conditional dispatch chain, as specified in section 2.c.

Listing 6: Transformed Scheme Program

```
(define pi 3.14)

(define square (lambda (x) (* x x)))

(define circle
  (lambda (x y radius)
    (lambda (msg)
      (if (eq? msg 'area)
          (* (square radius) pi)
```

```
            (if (eq? msg 'perimeter)
                (* 2 pi radius)
                'error)))))

(define c (circle 0 0 3))

(c 'area)
```

Explanation of Changes

- **Outer Lambda:** The `class (x y radius)` expression is transformed into an outer `lambda (x y radius)` which accepts the fields as arguments.
- **Inner Lambda:** It returns an inner closure `lambda (msg)` that acts as the object's method dispatcher.
- **Dispatch Chain:** Since the methods in this section have no parameters, the inner body evaluates directly to the first expression of each method's body based on the condition:
  - If `msg` is `'area`, it executes `(* (square radius) pi)`.
  - If `msg` is `'perimeter`, it executes `(* 2 pi radius)`.
  - Any unknown message falls back to the symbol `'error`.

Below is the chronological sequence of expressions passed as operands to the `L3applicativeEval` function during the execution of the transformed program under the Substitution Model.

1. **Evaluation of `pi` definition value:**

```
3.14
```

2. **Evaluation of `square` definition value:**

```
(lambda (x) (* x x))
```

3. **Evaluation of `circle` definition value:**

```
(lambda (x y radius)
  (lambda (msg)
    (if (eq? msg 'area)
        (* (square radius) pi)
        (if (eq? msg 'perimeter)
            (* 2 pi radius)
            'error))))
```

4. **Evaluation of the object `c` definition value `(circle 0 0 3)`:**

   - The entire application expression is evaluated:

```
(circle 0 0 3)
```

   - The operator and operands are evaluated individually:

```
circle
0
0
3
```

   - *Substitution Step:* The parameters `x` $\rightarrow$ `0`, `y` $\rightarrow$ `0`, and `radius` $\rightarrow$ `3` are substituted into the outer lambda's body. The resulting inner lambda expression is passed to evaluation:

```
(lambda (msg)
  (if (eq? msg 'area)
      (* (square 3) pi)
      (if (eq? msg 'perimeter)
          (* 2 pi 3)
          'error)))
```

5. **Evaluation of the final application `(c 'area)`:**

   - The entire application expression is evaluated:

```
(c 'area)
```

   - The operator and operands are evaluated individually:

```
c
'area
```

   - *Substitution Step:* The argument `msg` $\rightarrow$ `'area` is substituted into the closure's body. The resulting `if` expression is passed to evaluation:

```
(if (eq? 'area 'area)
    (* (square 3) pi)
    (if (eq? 'area 'perimeter)
        (* 2 pi 3)
        'error))
```

6. **Evaluation of the `if` condition predicate:**

   - The predicate expression is evaluated:

```
(eq? 'area 'area)
```

   - The operator and operands are evaluated individually:

```
eq?
'area
'area
```

7. **Evaluation of the `then` branch (since the predicate evaluates to `#t`):**

   - The combination is evaluated:

```
(* (square 3) pi)
```

   - The operator `*` is evaluated:

```
*
```

   - The first operand `(square 3)` is evaluated:

```
(square 3)
square
3
```

   - *Substitution Step:* The argument `x` $\rightarrow$ `3` is substituted into `square`'s body. The resulting expression is evaluated:

```
(* 3 3)
*
3
3
```

   - The second operand `pi` is evaluated:

```
pi
```